# Forms of Inheritance in Java

The following are the different forms of inheritance in java.

1. Specialization
2. Specification
3. Construction
4. Extension or Generalization
5. Limitation
6. Combination

1. **Specialization**

   It is the most ideal form of inheritance. The subclass is a special case of the parent class. It holds the principle of substitutability.

2. **Specification**

   This is another commonly used form of inheritance. In this form of inheritance, the parent class just specifies which methods should be available to the child class but doesn't implement them. The java provides concepts like abstract and interfaces to support this form of inheritance. It holds the principle of substitutability.

3. **Construction**

   This is another form of inheritance where the child class may change the behavior defined by the parent class (overriding). It does not hold the principle of substitutability.

4. **Extension or Generalization**

   This is another form of inheritance where the child class may add its new properties. It holds the principle of substitutability.

5. **Limitation**

   This is another form of inheritance where the subclass restricts the inherited behavior. It does not hold the principle of substitutability.

6. **Combination**

   This is another form of inheritance where the subclass inherits properties from multiple parent classes. Java does not support multiple inheritance type.

# Member access rules in Inheritance

1. ***public*** *Members*

- **Superclass**: public members of the superclass are accessible from any class, including subclasses, regardless of the package.
- **Subclass**: A subclass can access public members of its superclass without any restriction.

2. ***private*** *Members*

- **Superclass**: private members of the superclass **cannot** be accessed directly by any other class, including subclasses. These members are **hidden** from the subclass.
- **Subclass**: A subclass cannot access the private members of the superclass directly. However, the subclass can still access private members via **getter/setter methods** or other public/protected methods provided by the superclass.

3. ***protected*** *Members*

- **Superclass**: protected members of the superclass are accessible within the same package and in subclasses (even if they are in different packages).
- **Subclass**: A subclass can access protected members of its superclass, whether the subclass is in the same package or in a different package.

4. ***default*** *(Package-specific) Members*

- **Superclass**: Members with **default** access (no modifier) are accessible only within the same package.
- **Subclass**: If the subclass is in the same package, it can access these members. If the subclass is in a different package, it **cannot** access these members.

```
// Vehicle class with different access modifiers
class Vehicle
{
   //Can be accessed from anywhere
   public String brand;

   //Can only be accessed within the Vehicle class
   private int year;

   //Can be accessed within the package and by subclasses
   protected String model;

   //default Can be accessed only within the same package
   String color;
   public Vehicle(String brand, int year, String model, String color)
   {
      this.brand = brand;
      this.year = year;
      this.model = model;
      this.color = color;
   }
   // Public method to access the private variable
   public int getYear()
   {
      return year;
   }
   // Only accessible within the Vehicle class
   private void displayPrivateInfo()
   {
      System.out.println("Private method: Year of the vehicle is " + year);
   }
   //Accessible within the package and by subclasses
   protected void displayModel()
  {
      System.out.println("Model: " + model);
   }
}
```

```
// Subclass of Vehicle
class Car extends Vehicle
{
   public Car(String brand, int year, String model, String color)
   {
      super(brand, year, model, color);
   }
   // Subclass can access protected member and method
   public void displayCarInfo()
  {
      System.out.println("Brand: " + brand);
  System.out.println("Model (from subclass): " + model);  // access protected member
      displayModel();  // Can call protected method
   }
}

// Main class
public class Access
{
   public static void main(String[] args)
   {
      Vehicle v = new Vehicle("Toyota", 2020, "Corolla", "Red");
      // Public member can be accessed directly
      System.out.println("Brand: " + v.brand);
      // Cannot access private member directly, so using the public getter method
      System.out.println("Year: " + v.getYear());
      // Cannot access private method directly
      //v.displayPrivateInfo();  // This would cause a compile-time error
      // Accessing default/package-private member within the same package
      System.out.println("Color: " + v.color);
      // Subclass can access protected members and methods
      Car c = new Car("Honda", 2021, "Civic", "Blue");
      c.displayCarInfo();
   }
}
```

```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>javac Access.java

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java Access
Brand: Toyota
Year: 2020
Color: Red
Brand: Honda
Model (from subclass): Civic
Model: Civic

F:\VJIT\JAVA\LAB\Unit-II\PART 1>
```

# super keyword

The super keyword in Java is used as a **reference variable** to **access objects** from **parent classes**. It allows access to parent class constructors, members, and methods in the derived class. The keyword plays a crucial role in inheritance and polymorphism concepts.

**Use of super keyword in Java**

1. **super in Variables**: Used to access a variables from the superclass, especially if the subclass has a field with the same name.
2. **super in Methods**: Used to call a method from the superclass, especially when the method is overridden in the subclass.
3. **Super() in Constructors**: Used to invoke a constructor of the superclass. If the superclass doesn't have a no-argument constructor, you need to call one of its constructors explicitly using **super().**

```
//Program on Super Keyword
// Superclass
class Vehicle
{
   String brand;
   int year;
   // Constructor of the superclass
   Vehicle(String brand, int year)
  {
      this.brand = brand;
      this.year = year;
   }
   // Method in the superclass
   void displayInfo()
   {
      System.out.println("Brand: " + brand);
   }
}
// Subclass
class Car extends Vehicle
{
   int doors;
   // Constructor of the subclass
   Car(String brand, int year, int doors)
    {
      // Using super to call the superclass constructor
      super(brand, year);
      this.doors = doors;
     }
   // Method in the subclass
   void displayCarInfo()
   {
      // Using super to call the superclass method
      super.displayInfo();
         System.out.println("Year: " + super.year);
      System.out.println("Doors: " + doors);
   }
}
```

```
// Main class
public class SuperKey
{
   public static void main(String[] args)
  {
      Car c =new Car("Toyota", 2020, 4);
      c.displayCarInfo();
   }
}
```

```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>javac SuperKey.java

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java SuperKey
Brand: Toyota
Year: 2020
Doors: 4

F:\VJIT\JAVA\LAB\Unit-II\PART 1>
```

**Restrictions on super:**

- super can only be used to access members of the immediate superclass.
- super cannot be used to access private members of the superclass directly, as those are not visible outside the class.
- super cannot be used to call static methods or static variables of the superclass.

# Polymorphism

The term "polymorphism" means "many forms". In object-oriented programming, polymorphism is useful when you want to create multiple forms with the same name of a single entity.

Polymorphism in Java is mainly of **2 types** as mentioned below:

1. Method Overloading
2. Method Overriding

1. **Method Overloading:** Also, known as **compile-time** polymorphism, is the concept of Polymorphism where more than one method share the same name with different signature (Parameters) in a class. The return type of these methods can or cannot be same.

2. **Method Overriding:** Also, known as **run-time** polymorphism, is the concept of Polymorphism where method in the child class has the same name, return-type and parameters as in parent class. The child class provides the implementation in the method already written in the parent class.

```
//Program on method overloading & overriding
// Parent Class
class Parent
 {
   // Method Defined
   public void m1()
  {
      System.out.println("Parent Method m1 executed");
   }
   // Method Overloading
   public void m1(int a)
       {
      System.out.println("Parent Method m1 with parameter: " + a);
   }
}

// Child Class
class Child extends Parent
 {
   // Method Overriding
    public void m1(int a)
   {
          System.out.println("Child Method m1 with parameter: " + a);
   }
}
```

```
// Main Method
class Poly
 {
   public static void main(String args[])
   {
      Parent p = new Parent();
      p.m1();
      p.m1(5);
      Child c = new Child();
      c.m1(4);
   }
}
```

```
F:\VJIT\JAVA\LAB>java Poly
Parent Method m1 executed
Parent Method m1 with parameter: 5
Child Method m1 with parameter: 4

F:\VJIT\JAVA\LAB>_
```

```
//Program on method overriding
// Superclass vehicle
class Vehicle
{
   String brand;
   public Vehicle(String brand)
   {
      this.brand = brand;
   }

   // Method that will be overridden by subclasses
   public void hornSound()
   {
      System.out.println(brand + " horn sound: Beep Beep!");
   }
}
```

```
// Subclass of Vehicle
class Car extends Vehicle
{
    public Car(String brand)
    {
        super(brand);
    }
    // Overriding the hornSound method in the Car class
    public void hornSound()
    {
        System.out.println(super.brand +" horn sound: Honk Honk!");
    }
}
//Main class
public class Override
{
    public static void main(String[] args)
    {
        Vehicle v =new Vehicle("Toyota");
        v.hornSound();
        Car c =new Car("Honda");
        c.hornSound();
    }
}
```

Command Prompt

```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>javac Override.java

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java Override
Toyota horn sound: Beep Beep!
Honda horn sound: Honk Honk!

F:\VJIT\JAVA\LAB\Unit-II\PART 1>_
```

**Inheritance & Polymorphism** **PART-1**

***Inheritance:*** *Introduction, Forms of Inheritance - specialization, specification, construction, extension, limitation, combination, Member access rules, super keyword*
***Polymorphism-*** *method overriding, abstract classes, final keyword.*

# Inheritance

Inheritance is an important pillar of OOP (Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class.

In Java, Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class. In addition, you can add new fields and methods to your current class as well.

The class which inherits the properties of other is known as **subclass** (derived class, child class) and the class whose properties are inherited is known as **superclass** (base class, parent class).

It represents a **parent-child relationship** between two classes. This parent-child relationship is also known as an **IS-A** relationship.

**Need of Inheritance**

- **Code Reusability**: The basic need of an inheritance is to reuse the features. If you have defined some functionality once, by using the inheritance you can easily use them in other classes and packages.
- **Extensibility**: The inheritance helps to extend the functionalities of a class. If you have a base class with some functionalities, you can extend them by using the inheritance in the derived class.
- **Implementation of Method Overriding**: Inheritance is required to achieve one of the concepts of Polymorphism which is Method overriding.
- **Achieving Abstraction**: Another concept of OOPs that is abstraction also needs inheritance.

## Syntax to implement inheritance

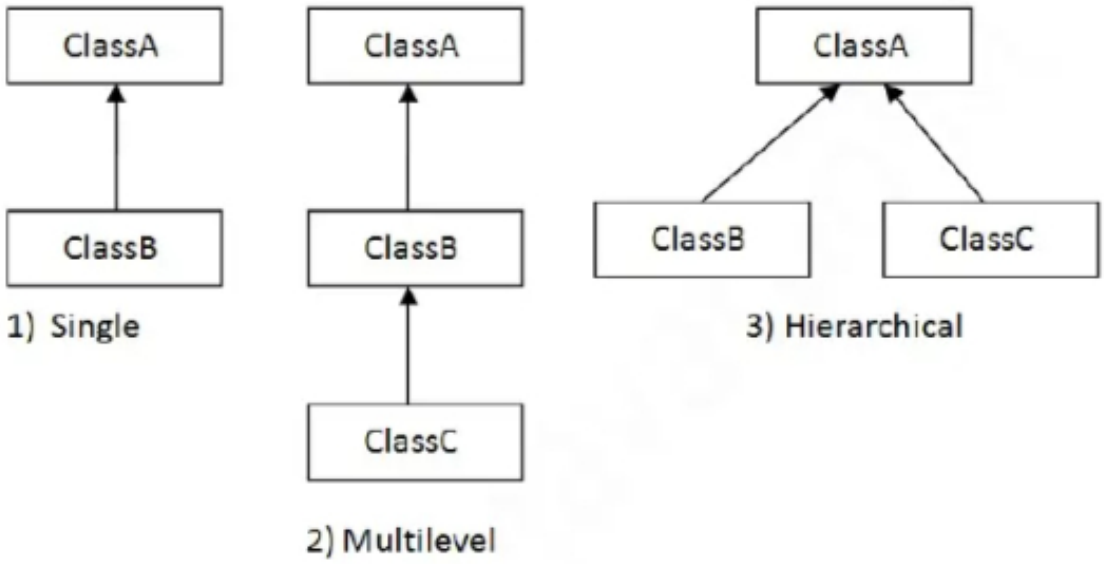
```
class Super
{
  .....
  .....
}
class Sub extends Super
{
  .....
  .....
}
```

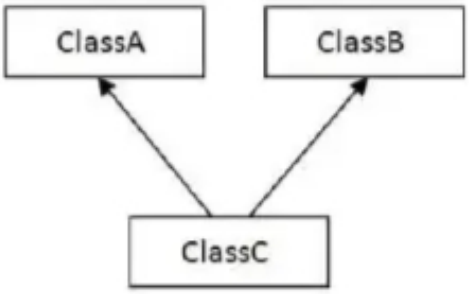
The **extends** keyword indicates that we are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

## Types of Inheritance in Java

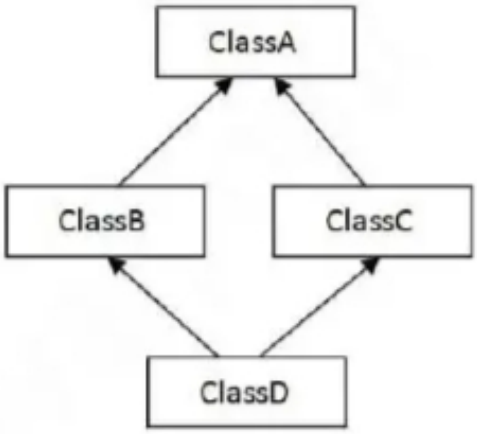
Five different types of inheritances are possible in Object-Oriented Programming.

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance






4) Multiple




5) Hybrid

**Note: Multiple inheritance is not supported in Java through class.**

When one class inherits multiple classes, it is known as multiple inheritance. For **Example:** there are three classes A, B, and C. The C class inherits A and B classes. If A and B classes have the same method and we call it from child class object, there will be ambiguity to call the method of A or B class.

1. **Single Inheritance:**

   In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance. In the below figure, 'A' is a parent class and 'B' is a child class. The class 'B' inherits all the properties of the class 'A'.

Single Inheritance

```
//Program on Single Inheritance
// Superclass: Vehicle
class Vehicle
{
    String brand;
    int speed;

    public Vehicle(String brand, int speed)
    {
        this.brand = brand;
        this.speed = speed;
    }
    public void displayInfo()
    {
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed + " km/h");
    }
}

// Subclass: Car
class Car extends Vehicle
{
    int numberOfDoors;

    public Car(String brand, int speed, int numberOfDoors)
      {
        super(brand, speed);
        this.numberOfDoors = numberOfDoors;
    }

    public void displayCarInfo()
      {
        displayInfo();
        System.out.println("Number of Doors: " + numberOfDoors);
    }
}
```

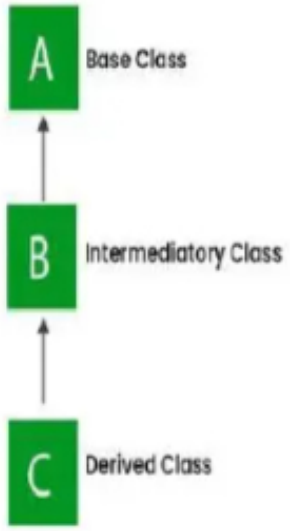
```
public class Single
{
   public static void main(String[] args)
     {
       Car c = new Car("Toyota", 180, 4);
       c.displayCarInfo();
   }
}
```

```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>javac Single.java

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java Single
Brand: Toyota
Speed: 180 km/h
Number of Doors: 4
```

## 2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the **grandparent's members.**




Multilevel Inheritance

```
//Program on MultiLevel Inheritance
// Superclass: Vehicle
class Vehicle
{
    String brand;
    public Vehicle(String brand)
    {
        this.brand = brand;
    }
    public void displayBrand()
    {
        System.out.println("Brand: " + brand);
    }
}
// Subclass: Car (derived from Vehicle)
class Car extends Vehicle
{
    int speed;
    public Car(String brand, int speed)
    {
        super(brand);
        this.speed = speed;
    }
    public void displaySpeed()
    {
        System.out.println("Speed: " + speed + " km/h");
    }
}
// Subclass: SportsCar (derived from Car)
class SportsCar extends Car
{
    boolean isTurbo;
    public SportsCar(String brand, int speed, boolean isTurbo)
    {
        super(brand, speed);
        this.isTurbo = isTurbo;
    }
```

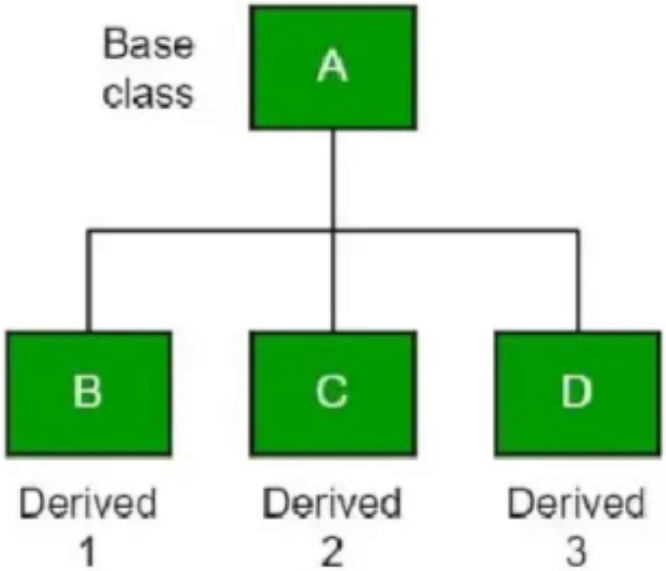
```
    public void displayTurbo()
    {
        System.out.println("Turbo: " + (isTurbo ? "Yes" : "No"));
    }
}
public class MultiLevel
{
    public static void main(String[] args)
    {
        SportsCar sc = new SportsCar("Ferrari", 350, true);
        sc.displayBrand();
        sc.displaySpeed();
        sc.displayTurbo();
    }
}
```

```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>java MultiLevel
Brand: Ferrari
Speed: 350 km/h
Turbo: Yes

F:\VJIT\JAVA\LAB\Unit-II\PART 1>
```

### 3. Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.



```
//Program on Hierarchical Inheritance
// Superclass: Vehicle
class Vehicle
{
   String brand;
   int speed;

   public Vehicle(String brand, int speed)
   {
      this.brand = brand;
      this.speed = speed;
   }

   public void displayInfo()
   {
      System.out.println("Brand: " + brand);
      System.out.println("Speed: " + speed + " km/h");
   }
}

// Subclass: Car
class Car extends Vehicle
{
   int numberOfDoors;

   public Car(String brand, int speed, int numberOfDoors)
   {
      super(brand, speed);
      this.numberOfDoors = numberOfDoors;
   }

   public void displayCarInfo()
      {
      displayInfo();
      System.out.println("Number of Doors: " + numberOfDoors);
   }
}
```

```
// Subclass: Truck
class Truck extends Vehicle
{
   int loadCapacity;
   public Truck(String brand, int speed, int loadCapacity)
        {
      super(brand, speed);
      this.loadCapacity = loadCapacity;
   }
   public void displayTruckInfo()
   {
      displayInfo();
      System.out.println("Load Capacity: " + loadCapacity + " tons");
   }
}
//Main Class
public class Hierarchical
{
   public static void main(String[] args)
        {
      Car c =new Car("Honda", 160, 4);
      Truck t = new Truck("Volvo", 120, 15);

      c.displayCarInfo();
      t.displayTruckInfo();
   }
}
```

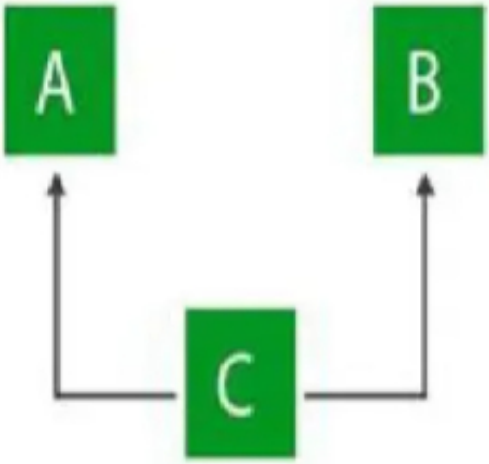
```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>javac Hierarchical.java

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java Hierarchical
Brand: Honda
Speed: 160 km/h
Number of Doors: 4
Brand: Volvo
Speed: 120 km/h
Load Capacity: 15 tons

F:\VJIT\JAVA\LAB\Unit-II\PART 1>
```

## 4. Multiple Inheritance (Through Interfaces)

In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. Please note that Java does **not** support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces. In the image below, Class C is derived from interfaces A and B.




Multiple Inheritance

```
//Program to implement Multiple Inheritance
// Interface 1: Speed
interface Speed
{
   void setSpeed(int speed);
   void displaySpeed();
}

// Interface 2: Fuel
interface Fuel
{
   void setFuelType(String fuelType);
   void displayFuelType();
}
```

```
// Concrete class: Vehicle implements both interfaces
class Vehicle implements Speed, Fuel
{
    String brand;
    int speed;
    String fuelType;

    public Vehicle(String brand)
    {
        this.brand = brand;
    }

    @Override
    public void setSpeed(int speed)
    {
        this.speed = speed;
    }

    @Override
    public void displaySpeed()
    {
        System.out.println("Speed: " + speed + " km/h");
    }

    @Override
    public void setFuelType(String fuelType)
    {
        this.fuelType = fuelType;
    }

    @Override
    public void displayFuelType()
    {
        System.out.println("Fuel Type: " + fuelType);
    }
```

```
public void displayBrand()
 {
      System.out.println("Brand: " + brand);
   }
}
//Main class
public class Multiple
{
   public static void main(String[] args)
 {
      Vehicle v = new Vehicle("Tesla");
      v.setSpeed(200);
      v.setFuelType("Electric");

      v.displayBrand();
      v.displaySpeed();
      v.displayFuelType();
   }
}
```

Command Prompt

```
F:\VJIT\JAVA\LAB\Unit-II\PART 1>javac Multiple.java

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java Multiple
Brand: Tesla
Speed: 200 km/h
Fuel Type: Electric

F:\VJIT\JAVA\LAB\Unit-II\PART 1>
```

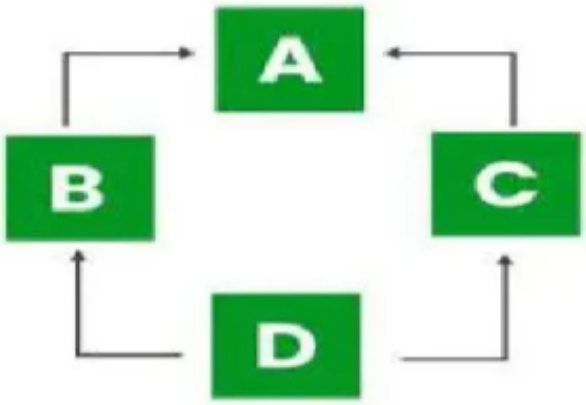
## 5. Hybrid Inheritance

It is a mix of two or more of the above types of inheritance. Since Java doesn't support multiple inheritances with classes, hybrid inheritance involving multiple inheritance is also not possible with classes. In Java, we can achieve hybrid inheritance only through **Interfaces** if we want to involve multiple inheritance to implement Hybrid inheritance.

However, it is important to note that Hybrid inheritance does not necessarily require the use of Multiple Inheritance exclusively.

- It can be achieved through a combination of Multilevel Inheritance and Hierarchical Inheritance with classes.
- Hierarchical and Single Inheritance with classes.

Therefore, it is indeed possible to implement Hybrid inheritance using classes alone, without relying on multiple inheritance type.




Hybrid Inheritance

```
//Program on Hybrid Inheritance
// Interface 1: Speed
interface Speed
{
   void setSpeed(int speed);
   void displaySpeed();
}

// Interface 2: Engine
interface Engine
{
   void startEngine();
   void stopEngine();
}
```

```
// Superclass: Vehicle
class Vehicle
{
    String brand;

    public Vehicle(String brand)
    {
        this.brand = brand;
    }

    public void displayBrand()
    {
        System.out.println("Brand: " + brand);
    }
}

// Subclass: Car implements both interfaces (Speed, Engine)
class Car extends Vehicle implements Speed, Engine
{
    int speed;

    public Car(String brand)
    {
        super(brand);
    }

    @Override
    public void setSpeed(int speed)
    {
        this.speed = speed;
    }

    @Override
    public void displaySpeed()
    {
        System.out.println("Speed: " + speed + " km/h");
    }
```

```
    @Override
    public void startEngine()
    {
        System.out.println("Engine started.");
    }

    @Override
    public void stopEngine()
    {
        System.out.println("Engine stopped.");
    }
}
//Main class
public class Hybrid
{
    public static void main(String[] args)
    {
        Car c = new Car("BMW");
        c.displayBrand();
        c.setSpeed(220);
        c.displaySpeed();
        c.startEngine();
        c.stopEngine();
    }
}
```

```
Command Prompt

F:\VJIT\JAVA\LAB\Unit-II\PART 1>java Hybrid
Brand: BMW
Speed: 220 km/h
Engine started.
Engine stopped.

F:\VJIT\JAVA\LAB\Unit-II\PART 1>
```